

Multithreading: Thread Lifecycle, Synchronization, ExecutorService

1. What is a Thread?

A **thread** is an independent path of execution inside a process.

A Java application can have multiple threads executing concurrently.

For example:

```
Spring Boot Application
|
+--- Thread 1 → Payment request A
|
+--- Thread 2 → Payment request B
|
+--- Thread 3 → Payment request C
```

In a web application, multiple incoming requests may be processed concurrently by different threads from the server's thread pool.

2. Process vs Thread

Process

A process is a running application with its own memory space.

Thread

A thread is an execution unit inside a process and shares process memory with other threads.

```
Process
├─ Thread 1
├─ Thread 2
└─ Thread 3
```

Because threads share memory, synchronization becomes necessary.

3. Thread Lifecycle

Java's `Thread.State` has these important states:

```
NEW
↓
RUNNABLE
↓
[executing / waiting for CPU]
```

↓
TERMINATED

And a thread can also enter:

BLOCKED
WAITING
TIMED_WAITING

NEW

Thread object has been created but hasn't started.

```
Thread t = new Thread(() -> {  
    System.out.println("Hello");  
});
```

At this point:

NEW

RUNNABLE

After:

```
t.start();
```

the thread enters `RUNNABLE`.

```
t.start();
```

Important interview point:

`start()` causes the JVM to schedule a new thread. It does **not** directly execute the `run()` method on the current thread.

4. `start()` vs `run()`

Very common question.

`start()`

```
t.start();
```

Creates/schedules a separate thread of execution.

`run()`

```
t.run();
```

is just a normal method call if invoked directly.

Example:

```
Thread t = new Thread(() ->
    System.out.println(
        Thread.currentThread().getName()));

t.run();
```

The code runs on the **current thread**, not a newly started thread.

Whereas:

```
t.start();
```

allows the new thread to execute.

5. BLOCKED

A thread becomes **BLOCKED** when it is waiting to acquire a monitor lock.

Example:

```
synchronized (lock) {
    ...
}
```

If Thread A owns the lock:

```
Thread A → owns lock
Thread B → waiting for lock
          ↓
        BLOCKED
```

6. WAITING

A thread enters **WAITING** when it waits indefinitely for another thread/action.

Examples:

```
Object.wait();
Thread.join();
```

Conceptually:

```
Thread A
  ↓
wait()

Thread B does something
  ↓
notify()/completes
  ↓
```

Thread A continues

7. TIMED_WAITING

The thread waits for a specified period.

Examples:

```
Thread.sleep(1000);
```

or:

```
object.wait(1000);
```

or:

```
thread.join(1000);
```

8. TERMINATED

The thread's execution has completed.

```
RUNNABLE
  ↓
run() completes
  ↓
TERMINATED
```

A thread cannot be restarted after termination.

```
t.start();
t.start(); // IllegalStateException
```

9. Why Synchronization?

Suppose two threads process the same payment simultaneously.

Shared variable:

```
int balance = 1000;
```

Thread A:

```
Read 1000
Subtract 500
Write 500
```

Thread B:

```
Read 1000
Subtract 700
Write 300
```

Final result might be:

```
300
```

when the business operation should actually detect that the operations conflict.

This is a **race condition**.

10. Race Condition

A race condition occurs when the result depends on the timing/interleaving of concurrent operations on shared state.

For a payment system:

```
Payment TX100
  ↓
Thread A → checks status = INITIATED
Thread B → checks status = INITIATED
  ↓
Both process payment
```

Potentially:

```
Payment processed twice
```

This is exactly why concurrency control matters in payment applications.

11. `synchronized`

Java provides the `synchronized` keyword to protect critical sections.

```
public synchronized void processPayment() {
    ...
}
```

or:

```
public void processPayment() {
    synchronized (this) {
        ...
    }
}
```

Only one thread at a time can execute the synchronized critical section for the same monitor.

12. Synchronized Method

```
public synchronized void increment() {  
    count++;  
}
```

For an instance method, the lock is typically the current object (`this`).

So:

```
Thread A  
  ↓  
lock(object)  
  ↓  
increment()  
  ↓  
unlock  
  
Thread B  
  ↓  
waits
```

13. Synchronized Block

Often more precise:

```
public void process() {  
    // non-critical code  
  
    synchronized (lock) {  
        // critical section  
    }  
  
    // more code  
}
```

This is often preferable because you can lock **only the shared critical section** rather than the entire method.

14. What is a Monitor Lock?

Every Java object can serve as a monitor associated with intrinsic synchronization.

When a thread enters:

```
synchronized (lock) {  
    ...  
}
```

it acquires the monitor associated with `lock`.

Other threads attempting to enter a synchronized section using the same lock have to wait.

15. Important synchronized Point

Suppose:

```
synchronized (lock1) {  
    ...  
}
```

Another thread can still execute:

```
synchronized (lock2) {  
    ...  
}
```

because the locks are different.

Therefore synchronization only coordinates threads that synchronize on the **same monitor/lock**.

16. Atomicity

Consider:

```
count++;
```

It looks like one operation, but conceptually it involves:

```
read  
+  
increment  
+  
write
```

Two threads can interleave those steps.

Therefore `count++` is not inherently atomic.

This is why:

```
AtomicInteger count = new AtomicInteger();  
  
count.incrementAndGet();
```

can be useful when you need an atomic counter.

17. volatile

Another common interview question.

```
private volatile boolean running = true;
```

`volatile` ensures visibility of updates across threads according to the Java Memory Model.

But:

`volatile` does **not** make compound operations such as `count++` atomic.

This is an important distinction.

```
volatile
→ visibility

synchronized / locks / atomic classes
→ synchronization/atomicity as appropriate
```

18. `synchronized` vs `volatile`

	<code>synchronized</code>	<code>volatile</code>
Visibility	Yes	Yes
Mutual exclusion	Yes	No
Protects critical section	Yes	No
Makes <code>count++</code> safe	Yes, if properly synchronized	No

19. Deadlock

Very important.

Deadlock happens when threads wait for locks held by each other indefinitely.

Example:

```
Thread A
owns Lock 1
waits for Lock 2
```

```
Thread B
owns Lock 2
waits for Lock 1
```

```
A → Lock1 → waiting for Lock2
B → Lock2 → waiting for Lock1
```

Neither can proceed.

Prevention

Common approaches:

- consistent lock ordering
 - avoid unnecessary nested locks
 - use `Lock.tryLock()` with timeout where appropriate
 - minimize critical sections
-

20. ExecutorService

This is one of the **highest-priority topics** for your interview.

Instead of manually creating threads:

```
new Thread(...).start();
```

you generally use a **thread pool** for application-level concurrent task execution.

```
ExecutorService executor =  
    Executors.newFixedThreadPool(5);
```

Then:

```
executor.submit(() -> {  
    processPayment();  
});
```

21. Why ExecutorService?

Imagine 10,000 tasks arrive.

Creating a new thread for every task:

```
Task 1 → Thread 1  
Task 2 → Thread 2  
Task 3 → Thread 3  
...  
Task 10000 → Thread 10000
```

is expensive and can exhaust system resources.

With a thread pool:

```
10,000 tasks  
↓  
Task Queue  
↓  
Thread Pool  
├─ T1  
├─ T2  
├─ T3  
├─ T4  
└─ T5
```

Threads are reused.

Benefits:

- limits concurrency
 - reduces thread creation overhead
 - manages task queueing
 - improves resource utilization
-

22. Common Executor Types

Fixed Thread Pool

```
ExecutorService executor =  
    Executors.newFixedThreadPool(10);
```

A fixed number of worker threads.

Useful when you want to control concurrency.

Single Thread Executor

```
ExecutorService executor =  
    Executors.newSingleThreadExecutor();
```

One worker thread processes submitted tasks sequentially.

Scheduled Executor

```
ScheduledExecutorService executor =  
    Executors.newScheduledThreadPool(2);
```

Can execute tasks later or periodically.

Cached Thread Pool

```
Executors.newCachedThreadPool();
```

Can create threads as needed and reuse idle ones.

Use carefully; unbounded growth can become problematic under heavy load.

23. `submit()` vs `execute()`

Common interview question.

`execute()`

```
executor.execute(() -> process());
```

Returns nothing.

```
void
```

`submit()`

```
Future<?> future =
```

```
executor.submit(() -> process());
```

Returns a `Future`.

You can use it to:

- check completion
- retrieve a result
- catch task exceptions through `Future.get()`
- cancel the task

24. Callable vs Runnable

Runnable

Doesn't return a result.

```
Runnable task = () -> {  
    processPayment();  
};
```

Callable

Returns a result and can throw checked exceptions.

```
Callable<PaymentResponse> task = () -> {  
    return processPayment();  
};
```

Then:

```
Future<PaymentResponse> future =  
    executor.submit(task);  
  
PaymentResponse response =  
    future.get();
```

25. What is a Future?

`Future` represents the result of an asynchronous computation.

```
submit task  
  ↓  
Future  
  ↓  
task executes  
  ↓  
result available
```

Example:

```
Future<String> future =  
    executor.submit(() -> "SUCCESS");
```

```
String result = future.get();
```

Be careful:

```
future.get();
```

is blocking if the result isn't ready.

26. ExecutorService Lifecycle

You should shut down the executor when you're responsible for its lifecycle.

```
executor.shutdown();
```

This means:

Stop accepting new tasks, but allow already submitted tasks to finish.

`shutdownNow()` attempts to stop running tasks by interrupting threads and returns tasks that were waiting in the queue. It does not guarantee that running tasks will immediately stop.

27. Spring Boot and Thread Pools

In Spring applications, you may encounter:

```
@Async
```

Example:

```
@Async
public void sendNotification() {
    ...
}
```

But don't assume `@Async` automatically gives unlimited resources. It uses a configured executor.

A custom executor can be configured:

```
@Bean
public Executor paymentExecutor() {

    ThreadPoolTaskExecutor executor =
        new ThreadPoolTaskExecutor();

    executor.setCorePoolSize(5);
    executor.setMaxPoolSize(10);
    executor.setQueueCapacity(100);
    executor.setThreadNamePrefix("payment-");

    executor.initialize();

    return executor;
}
```

Then:

```
@Async("paymentExecutor")
public void processAsync() {
    ...
}
```

28. Important Spring @Async Question

Suppose:

```
@Service
public class PaymentService {

    public void methodA() {
        methodB();
    }

    @Async
    public void methodB() {
        ...
    }
}
```

Will `methodB()` necessarily execute asynchronously?

Not when called through `this.methodB()` inside the same bean.

Spring's `@Async` is proxy-based, so self-invocation bypasses the proxy.

This is a strong 3-year Spring question.

29. Concurrency and Your Payment Application

This is where you should connect the topic to your project.

Suppose:

```
Transaction TX100
  ↓
Request A
  +
Request B
```

arrive at nearly the same time.

Both execute:

```
Check:
status == INITIATED?
```

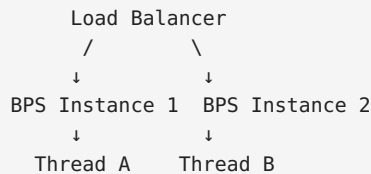
Both see:

INITIATED

Then both process the payment.

A Java `synchronized` block **inside one application instance is not necessarily sufficient**, especially when your application runs multiple instances.

Example:



A JVM-local lock can't coordinate both instances.

This is where you need system-level mechanisms such as:

- database constraints
- optimistic/pessimistic locking
- idempotency keys
- distributed locking where justified
- transactional state transitions

This is an important architecture-level distinction.

30. Optimistic Locking Example

A payment record might have:

```
id = TX100
version = 5
status = INITIATED
```

Two transactions read version 5.

Thread A updates:

```
version 5 → 6
```

Thread B then tries to update based on version 5.

The database detects the version mismatch and rejects the stale update.

In JPA:

```
@Version
private Long version;
```

This is frequently relevant to concurrent business updates.

31. `synchronized` vs Database Locking

Don't confuse JVM-level synchronization with database concurrency control.

synchronized

Protects shared state inside one JVM

Database locking

Coordinates transactions at the database level

For payment transaction state, database-level concurrency control is often much more relevant than simply putting `synchronized` around a service method.

32. ExecutorService Best Practices

Don't blindly create:

```
Executors.newFixedThreadPool(1000);
```

You should consider:

- CPU vs I/O workload
- expected concurrency
- queue size
- task duration
- downstream capacity
- memory usage
- rejection policy
- monitoring

A thread pool is a **resource-management decision**, not just a syntax choice.

33. CPU-bound vs I/O-bound

CPU-bound

Examples:

```
Encryption  
Complex calculations  
Data processing
```

Too many threads can cause excessive context switching.

I/O-bound

Examples:

```
DB calls  
HTTP calls
```

Threads may spend significant time waiting, so more concurrency may be useful—but the real bottleneck may be the downstream system.

For payment applications:

```
BPS
↓
HTTP gateway
↓
Network wait
```

is often I/O-heavy.

But you still need to respect the gateway's capacity and your application's resource limits.

34. Modern Java note

If your interview involves **Java 21+**, you should also know **virtual threads**.

Java provides:

```
Thread.ofVirtual()
```

and executors such as:

```
Executors.newVirtualThreadPerTaskExecutor()
```

Virtual threads are designed to make large numbers of concurrent blocking I/O tasks more efficient.

However, don't automatically replace every existing thread pool with virtual threads. Application architecture, blocking behavior, resource limits, synchronization, database connection pools, and downstream capacity still matter.

35. High-Priority Interview Questions

Thread basics

1. What is a thread?
2. Process vs thread?
3. Explain thread lifecycle.
4. `start()` vs `run()` ?
5. What are `BLOCKED`, `WAITING`, and `TIMED_WAITING` ?
6. Can a terminated thread be restarted?

Synchronization

7. What is a race condition?
8. What is synchronization?
9. What does `synchronized` do?
10. Synchronized method vs block?

11. What is a monitor lock?
12. What is deadlock?
13. How can deadlock be prevented?
14. What is `volatile`?
15. `volatile` vs `synchronized`?
16. Why isn't `count++` thread-safe?
17. What is `AtomicInteger`?

ExecutorService

18. Why use `ExecutorService`?
19. What is a thread pool?
20. `execute()` vs `submit()`?
21. `Runnable` vs `Callable`?
22. What is `Future`?
23. What happens when `Future.get()` is called?
24. `shutdown()` vs `shutdownNow()`?
25. Fixed thread pool vs cached thread pool?
26. How do you size a thread pool?

Spring

27. How does `@Async` work?
28. Why can self-invocation break `@Async`?
29. How do you configure a custom executor?
30. What happens if the executor queue becomes full?

Project-level

31. Where did you use asynchronous processing?
32. Why did you need multiple threads?
33. How did you prevent duplicate processing?
34. What happens when two requests update the same payment?
35. Would `synchronized` solve the problem?
36. How would it behave with multiple application instances?
37. Would you use optimistic or pessimistic locking?
38. How would you prevent a thread pool from overwhelming a downstream payment system?

36. Interview-ready answers

Q: Why do we need synchronization?

“When multiple threads access shared mutable state concurrently, their operations can interleave and produce incorrect results. Synchronization provides mutual exclusion around critical sections and also establishes the necessary memory-visibility guarantees.”

Q: Why use `ExecutorService` instead of creating threads manually?

“`ExecutorService` provides thread-pool management. Instead of creating a new thread for every task, we can reuse a controlled number of worker threads, queue tasks, manage concurrency, and obtain task results through `Future` when needed.”

Q: Is `synchronized` enough to prevent duplicate payment processing?

“Not necessarily. `synchronized` only coordinates threads sharing the same JVM monitor. In a horizontally scaled application with multiple instances, separate JVMs don’t share that lock. For payment processing I would use application-level idempotency and database concurrency controls appropriate to the transaction.”

That last answer is particularly valuable for **your BPS project** because it connects Java multithreading knowledge to real distributed-system behavior.